Deploy y Releases — Kumi TTPN

Fecha: 2026-04-11
Audiencia: Dev que hace el deploy, Tech Lead que aprueba

Repositorios y Remotes

Backend (`ttpngas`)

Remote	URL	Uso
`origin`	`git@gitlab.com:ttpn/ttpngas.git`	Fuente de verdad principal
`github`	`github.com/webmasterttpn/kumi-admin-api`	Espejo + SonarCloud CI
`heroku`	`git.heroku.com/ttpngas.git`	Legacy — **no usar para deploys nuevos**

Rama de producción: `transform_to_api`
Railway escucha: la rama `transform_to_api` del remote `github`

Frontend (`ttpn-frontend`)

Remote	URL	Uso
`origin`	`gitlab.com/TTPN_Antonio/ttpn-frontend`	Fuente de verdad
`github`	`github.com/webmasterttpn/kumi-admin-frontend`	Espejo + Netlify CI/CD

Rama de producción: `main`
Netlify escucha: la rama `main` del remote `github`

Flujo de Ramas

```
feature/nombre-de-la-tarea
        │
        │  Pull Request / merge manual
        ▼
  transform_to_api  (BE)  /  main  (FE)
        │
        │  push a github remote
        ▼
  Railway auto-deploy (BE)  /  Netlify auto-deploy (FE)
```

Convención de ramas

Prefijo	Cuándo usarlo
`feature/`	Feature nueva (ej. `feature/mobile-auth-jwt`)
`fix/`	Bug fix (ej. `fix/payroll-id-null`)
`chore/`	Mantenimiento, deps, docs (ej. `chore/update-gems`)
`hotfix/`	Fix urgente directo a producción

Deploy del Backend (Railway)

Railway hace **auto-deploy** cada vez que hay un push a `transform_to_api` en el remote `github`.

Deploy normal (feature → producción)

```
# 1. Asegurarse de estar en la rama correcta y al día
cd ttpngas
git checkout transform_to_api
git pull origin transform_to_api

# 2. Mergear la feature terminada
git merge feature/mi-feature
# o si viene de GitLab, hacer el merge desde la UI de GitLab primero
# y luego:
git pull origin transform_to_api

# 3. Correr tests antes de subir
docker compose exec api bundle exec rails spec
# (si el proyecto tiene specs — actualmente en construcción)

# 4. Push al remote github → dispara el deploy en Railway
git push github transform_to_api

# 5. Push al remote origin (GitLab) para mantener sincronía
git push origin transform_to_api
```

Verificar el deploy

```
# Seguir los logs en Railway mientras despliega
railway logs --service api -f

# Señales de deploy exitoso en los logs:
# "Puma starting in single mode..."
# "Listening on http://0.0.0.0:3000"
# "Use Ctrl-C to stop"

# Health check manual
curl https://TU_DOMINIO.railway.app/up
# Respuesta esperada: "API TTPN Online" con 200
```

Tiempo estimado de deploy

Etapa	Tiempo
Build de imagen Docker	2-4 min
Arranque de Puma	30-60 seg
Total hasta servir requests	~3-5 min

Migraciones en Producción

Regla principal: Las migraciones **no corren automáticamente** en Railway.
Deben ejecutarse manualmente después de que el deploy terminó.

Procedimiento

```
# 1. Verificar qué migraciones están pendientes ANTES de hacer push
cd ttpngas
docker compose exec api bundle exec rails db:migrate:status | grep down

# 2. Hacer el deploy (push a github)
git push github transform_to_api

# 3. Esperar a que Railway termine el deploy
railway logs --service api -f
# Esperar hasta ver "Puma starting..."

# 4. Correr las migraciones en producción
railway run --service api bundle exec rails db:migrate

# 5. Verificar que no quedaron migraciones pendientes
railway run --service api bundle exec rails db:migrate:status | grep down
# No debe haber ninguna "down"

# 6. Si la migración modifica travel_counts o sus triggers:
# Verificar que el cuadre sigue funcionando (ver RUNBOOK sección 6)
```

Migraciones con triggers PG (máximo cuidado)

Las migraciones que reemplazan funciones PG (`sp_tctb_insert`, `buscar_booking`, etc.) son las más delicadas. Antes de correrlas en producción:

```
# Verificar los triggers activos antes de migrar
railway run --service api bundle exec rails runner "
result = ActiveRecord::Base.connection.execute(
  \"SELECT trigger_name FROM information_schema.triggers WHERE
```

```
trigger_schema='public'\"
)
result.each { |r| puts r['trigger_name'] }
"

# Después de migrar, verificar que los triggers siguen activos
# (mismo comando de arriba)

# Crear un travel_count de prueba y verificar que viaje_encontrado se
procesa
# Si el cuadre falla → hacer rollback inmediato (ver RUNBOOK sección 14)
```

Rollback de migración

```
# Rollback de la última migración
railway run --service api bundle exec rails db:rollback

# Rollback de N migraciones
railway run --service api bundle exec rails db:rollback STEP=2
```

Deploy del Frontend (Netlify)

Netlify hace **auto-deploy** cada vez que hay un push a `main` en el remote `github`.

Deploy normal

```
# 1. Estar en main y al día
cd ttpn-frontend
git checkout main
git pull origin main

# 2. Mergear la feature
git merge feature/mi-feature-fe

# 3. Push a github → dispara Netlify
git push github main

# 4. Push a origin (GitLab) para sincronía
git push origin main
```

Verificar el deploy

```
Netlify Dashboard → tu sitio → Deploys → ver el deploy en curso
Build exitoso: aparece como "Published" en verde
Build fallido: aparece en rojo con log del error
```

Señales de build exitoso en Netlify:

```
[Quasar] Compiling...
[Quasar] Build Successful
Netlify: Build script success
Deploy live in X.X seconds
```

Tiempo estimado

Etapa	Tiempo
Build Quasar/Vite	1-3 min
Deploy a CDN	30 seg
Total	~2-4 min

Variables de entorno en Netlify

Si el deploy falla por variable faltante:

```
Netlify Dashboard → Site Configuration → Environment Variables
Variables requeridas:
  API_URL = https://TU_DOMINIO.railway.app
```

Después de cambiar una variable: **Trigger deploy** manualmente desde Netlify.

Hotfix (fix urgente en producción)

Para bugs críticos que no pueden esperar el ciclo normal:

```
# Backend
git checkout transform_to_api
git pull origin transform_to_api
# Hacer el fix directamente en la rama de producción
git add .
git commit -m "hotfix: descripción del fix urgente"
git push github transform_to_api   # deploy inmediato
git push origin transform_to_api   # sincronizar GitLab

# Frontend
git checkout main
git pull origin main
# Hacer el fix
git add .
git commit -m "hotfix: descripción del fix urgente"
```

```
git push github main   # deploy inmediato
git push origin main
```

Sincronización de Remotes

El flujo normal empuja a dos remotes. Si quedaron desincronizados:

```
# Backend — sincronizar GitLab con GitHub
cd ttpngas
git fetch github
git checkout transform_to_api
git reset --hard github/transform_to_api
git push origin transform_to_api --force-with-lease
# --force-with-lease es más seguro que --force: falla si alguien
# hizo push a origin desde la última sincronización

# Frontend — sincronizar GitLab con GitHub
cd ttpn-frontend
git fetch github
git checkout main
git reset --hard github/main
git push origin main --force-with-lease
```

CI/CD Automático

SonarCloud (Backend)

Se ejecuta automáticamente en cada push a `transform_to_api` y en pull requests.

```
# .github/workflows/build.yml
on:
  push:
    branches: [transform_to_api]
  pull_request:
    types: [opened, synchronize, reopened]
```

Si SonarCloud falla:

- Ver el reporte en sonarcloud.io → proyecto `kumi-admin-api`
- Los issues de Quality Gate no bloquean el deploy en Railway — son informativos
- Pero deben resolverse antes del siguiente ciclo de features

No hay pipeline de tests automáticos aún

Los tests (RSpec) se corren manualmente antes de hacer push:

```
docker compose exec api bundle exec rails spec
```

Pendiente: agregar el job de tests al workflow de GitHub Actions.

Checklist de Release

Antes de cada release significativo (nueva feature, módulo completo):

Backend

- [ ] Todos los commits en `transform_to_api` (o mergeados desde feature branch)
- [ ] `db:migrate:status` no muestra migraciones pendientes en local
- [ ] El servidor local responde en `/up`
- [ ] Si hay migraciones que tocan triggers: probar cuadre manual con travel_count de prueba
- [ ] Push a `github` remote → esperar deploy Railway
- [ ] `rails db:migrate` en producción si hay migraciones nuevas
- [ ] Health check de producción: `curl https://dominio.railway.app/up`
- [ ] Login manual en el FE con usuario real
- [ ] Push a `origin` (GitLab) para sincronizar

Frontend

- [ ] Todos los cambios en `main`
- [ ] `quasar build` sin errores en local (o que el build de Netlify pase)
- [ ] Verificar en navegador que la versión desplegada tiene los cambios
- [ ] Si se cambió el contrato de API (nuevo endpoint, campo nuevo): verificar que el BE ya está en producción primero
- [ ] Push a `origin` (GitLab) para sincronizar

Post-release

- [ ] Actualizar la versión en Settings → Versiones de la Aplicación
- [ ] Si hay migración que afectó `travel_counts`: verificar cuadre del día (`TravelCount.where(fecha: Date.today).where(viaje_encontrado: true).count`)
- [ ] Notificar al equipo operativo si el release cambia algún flujo de trabajo

---

Versiones de la Aplicación

Las versiones visibles al usuario se registran en `Settings → Versiones`:

```
FE → /settings → Organización → Versiones de la Aplicación
```

Flujo para cambiar versión activa:

1. Editar la versión actual → poner `fecha_fin` futura (ej. en 30 días)

2. Crear nueva versión con `fecha_inicio` = hoy, sin `fecha_fin` (definitiva)

El sistema desactiva automáticamente las versiones expiradas cada noche a las 2am (`DeactivateExpiredVersionsJob`).

Resumen Visual

```
DEV LOCAL (Docker)
    │
    │  git commit + push
    ▼
GITLAB (origin)  ─── espejo ──►  GITHUB (github remote)
                                        │
                                ┌───────┴────────┐
                                ▼                ▼
                    RAILWAY (BE)            NETLIFY (FE)
                    auto-deploy             auto-deploy
                    transform_to_api        main
                              │
                              ▼ (manual)
                    rails db:migrate
                    en producción
```